Utils: PEB walking + resolver dinámico (GetModuleHandle/GetProcAddress)

> Objetivo: Entender y replicar en Rust el flujo clásico para:

- Encontrar módulos cargados recorriendo el **PEB**
- Resolver exports (funciones) leyendo el **PE/Export Directory**

Codigo utilizado:

- `crates/utils/src/win_types.rs`
- `crates/utils/src/lib.rs`

Primitives Windows -> Rust

Windows primitives	Rust primitives
CHAR	u8 or i8
UCHAR	u8
SHORT	i16
USHORT	u16
DWORD	u32
DWORDLONG	u64
DWORD64	u64
DWORD32	u32
WORD	u16
LONG	i32
ULONG	u32
PVOID HANDLE	*const c_void* mut c_void
LPCSTR	*const u8 or i8
LPWSTR	*const u8..u16..i8..i16

Notas:

- `LPWSTR` en Windows es UTF-16; en Rust normalmente lo representas como `*const u16`.
- `usize` es el tamaño nativo (x64 => 8 bytes) y te sirve como “portátil” para direcciones. Es decir si el binario es x86 el usize sera 4 bytes, pero si el binario es x64 el usize sera 8 bytes. **La misma cantidad de bytes que una direccion de memoria.

---

1) Estructuras

1.1 `#[repr(C)]`

En C no piensas en el layout, viene “por defecto”. En Rust, **si estás mapeando structs de Windows** debes usar `#[repr(C)]`.

Ejemplo (`crates/utils/src/win_types.rs`):

```
#[repr(C)]
pub struct LIST_ENTRY {
    pub Flink: *const LIST_ENTRY,
    pub Blink: *const LIST_ENTRY,
}
```

1.2 `unsafe` para manejar raw pointers.

En C es implícito, es decir siempre estas manejando raw pointers. En Rust, se vuelve explícito con `unsafe { ... }`.

Regla práctica:

- Practicamente siempre que queramos hacer algo con estructuras internas deberemos utilizar `unsafe`

---

2) PEB walking: Estructuras importantes

2.1 `UNICODE_STRING`

(`crates/utils/src/win_types.rs`):

Unicode strings y `u16` slices en Rust

1. Strings en C vs Unicode

En **C**, el tipo más común para strings es `char`:

```
char *str = "hello";
```

- Cada `char` ocupa **1 byte**
- El string es simplemente un **array de bytes**

```
"h"  "e"  "l"  "l"  "o"
68   65   6C   6C   6F
^    ^    ^    ^    ^
1B   1B   1B   1B   1B
```

Total: **5 bytes**

---

2. Cómo funciona Unicode en Windows

Muchas APIs de Windows utilizan **UTF-16**.

Esto significa que:

- Cada carácter ocupa **2 bytes**
- Los strings realmente son **arrays de** `u16`

Ejemplo:

```
H  e  l  l  o
0048 0065 006C 006C 006F
```

En memoria:
`48 00 65 00 6C 00 6C 00 6F 00`
Total: **10 bytes**
Pero **solo hay 5 caracteres**.

---

3. La estructura `UNICODE_STRING`

En Windows muchas funciones usan la estructura

La estructura en C tenemos:

```c
typedef struct _UNICODE_STRING {
    USHORT Length;
    USHORT MaximumLength;
    PWSTR  Buffer;
} UNICODE_STRING;
```

Estructura en Rust:

```rust
#[repr(C)]
pub struct UNICODE_STRING {
    pub Length: u16,
    pub MaximumLength: u16,
    pub Buffer: *const u16,
}
```

Campos importantes:

- **Length** → tamaño **en bytes**
- **Buffer** → puntero al string UTF-16

Ejemplo:

Length = 16 bytes

Pero como cada carácter ocupa **2 bytes**:

16 / 2 = 8 caracteres

---

4. El problema en Rust

En Rust no existe un `String` UTF-16 nativo como tipo común (fuera de ciertas utilidades de la librería estándar).

Por lo tanto, cuando trabajamos con APIs de Windows debemos **interpretar manualmente la memoria como** `u16`.

---

5. Convertir bytes a caracteres

Si `Length` está en **bytes**, primero debemos calcular cuántos **u16** hay.

```
cantidad_de_u16 = length / 2
```

Ejemplo:

Length = 16 bytes
16 / 2 = 8 u16

---

6. Crear un slice en Rust

Si tenemos un puntero:

```
let estructura : *const UNICODE_STRING;

//Pero len debe ser el número de elementos, no el número de bytes.
let len = length / 2;

// Podemos crear un **slice** usando:
let slice = core::slice::from_raw_parts(estructura.Buffer, len);
```

Resultado:

[u16; 8]

Este slice ocupa:

`8 * 2 bytes = 16 bytes`

Exactamente el tamaño original.

```
// Creamos una implementacio
impl UNICODE_STRING {

    pub fn to_string(&self) -> Option<String> {
        if self.Buffer as usize == 0 || self.Length as usize == 0 {
            println!("This is not even a Unicode String boy");
            return None;
        }

        let len: usize = (self.Length / 2) as usize;
        let slizi: &[u16] = unsafe { core::slice::from_raw_parts(self.Buffer,
len) };
```

```
// Retornamos un String luego de decodear el utf16
        Some(
            core::char::decode_utf16(slizi.iter().cloned())
                .collect::<Result<String, _>>()
                .ok()
                .unwrap(),
        )

    }

}
```

- `Length` está en **bytes**, por eso `self.Length / 2`.
- El “slice” se crea con `from_raw_parts` dentro de `unsafe`.

2.2 `PEB` / `PEB_LDR_DATA`

(`crates/utils/src/win_types.rs`):

```
#[repr(C)]

pub struct PEB_LDR_DATA {
    pub Reserved1: [u8; 8],
    pub Reserved2: [*const c_void; 3],
    pub InMemoryOrderModuleList: LIST_ENTRY,

}

pub type PPS_POST_PROCESS_INIT_ROUTINE = Option<unsafe extern "system" fn()>;


#[repr(C)]
pub struct PEB {
    pub Reserved1: [u8; 2],
    pub BeingDebugged: u8,
    pub Reserved2: u8,
    pub Reserved3: *const c_void,
    pub ImageBaseAddress: *const c_void,
    pub Ldr: *const PEB_LDR_DATA,
    pub ProcessParameters: RTL_USER_PROCESS_PARAMETERS,
    pub Reserved4: [*const c_void; 3],
```

```
    pub AtlThunkSListPtr: *const c_void,
    pub Reserved5: *const c_void,
    pub Reserved6: u32,
    pub Reserved7: *const c_void,
    pub Reserved8: u32,
    pub AtlThunkSListPtr32: u32,
    pub Reserved9: [*const c_void; 45],
    pub Reserved10: [u8; 96],
    pub PostProcessInitRoutine: PPS_POST_PROCESS_INIT_ROUTINE,
    pub Reserved11: [u8; 128],
    pub Reserved12: *const c_void,
    pub SessionId: u32,
}
```

---

3) PEB walking: obtener PEB con `GS:[0x60]`

(`crates/utils/src/lib.rs`):

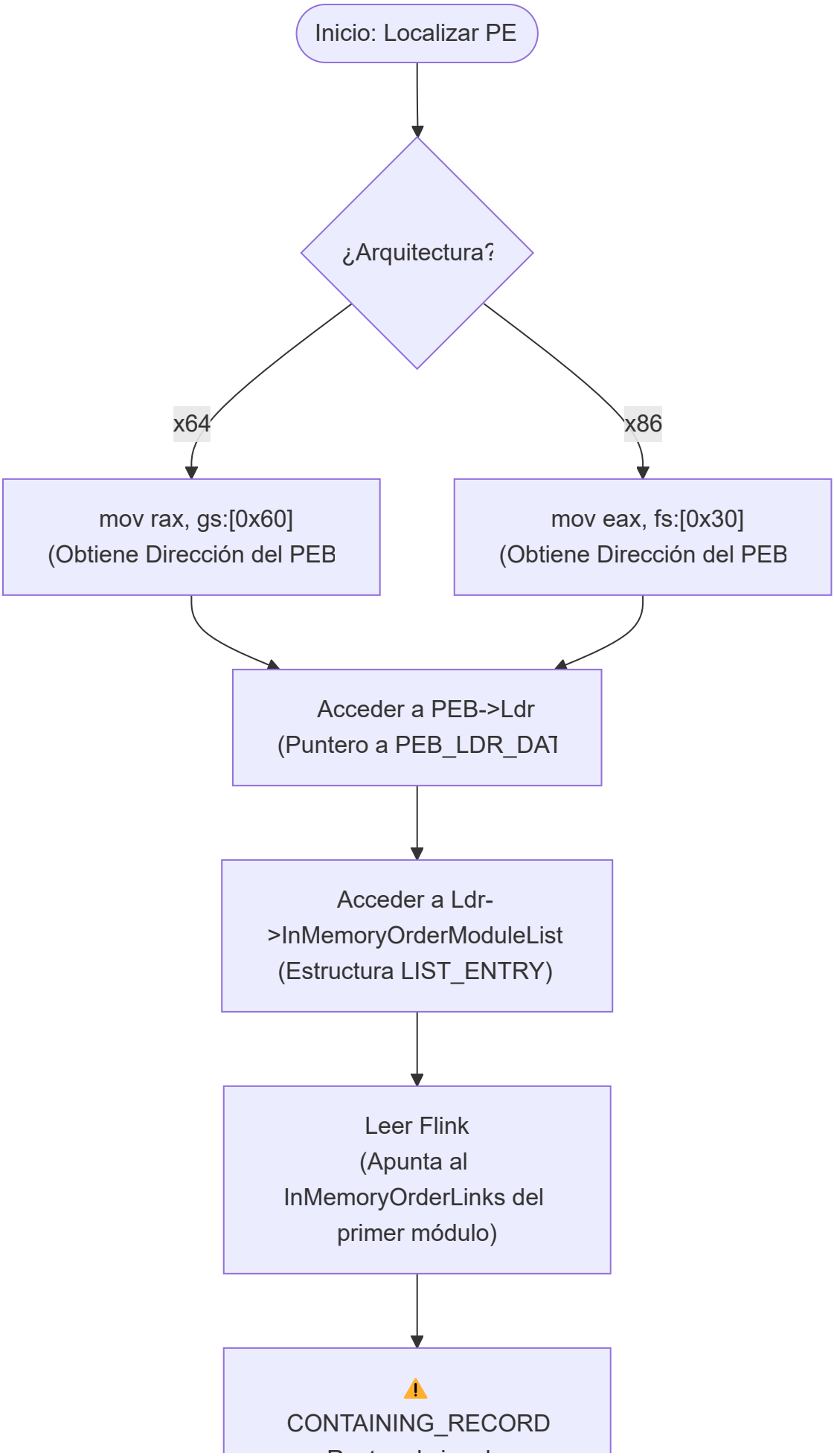
```
///It just read the gs[0x60] register

fn __readgsqword(offset: u64) -> u64{
    let out: u64;
    unsafe {asm!(
        "mov {}, gs:[{}]",
        out(reg) out,
        in(reg) offset,
    );

}
    return out;
}
```

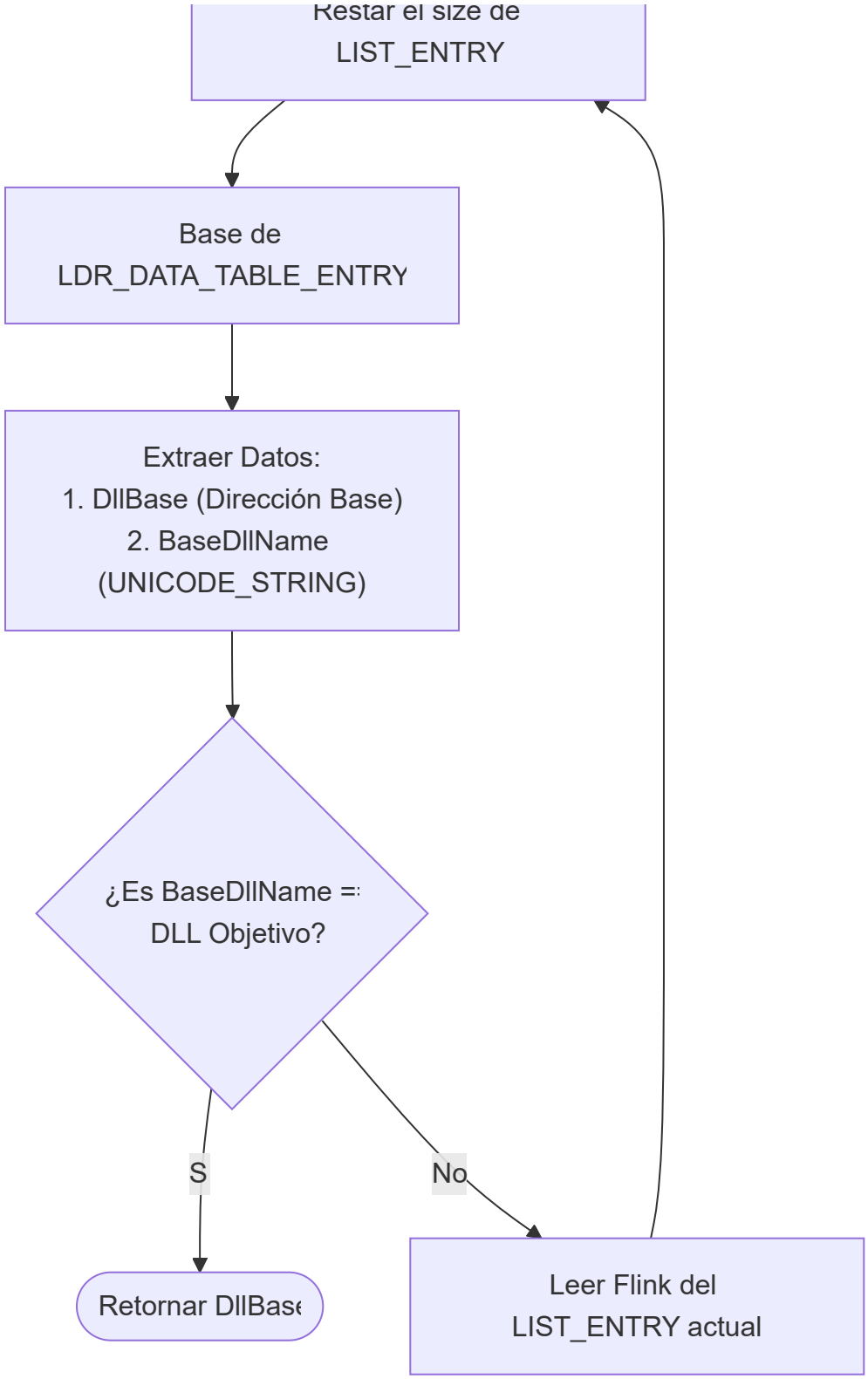
Diferencias a explicar:

- En C harías intrinsics/asm similar, pero en Rust esto queda confinado a `unsafe { asm! (...) }`
- Esto asume **x64** y no x86.

---

4) PEB walking: `GetModuleHandle`


Inicio: Localizar PE
¿Arquitectura?
x64
x86
mov rax, gs:[0x60]
(Obtiene Dirección del PEB
mov eax, fs:[0x30]
(Obtiene Dirección del PEB
Acceder a PEB->Ldr
(Puntero a PEB_LDR_DAT
Acceder a Ldr->InMemoryOrderModuleList
(Estructura LIST_ENTRY)
Leer Flink
(Apunta al InMemoryOrderLinks del primer módulo)
⚠️
CONTAINING_RECORD



Este repo implementa `GetModuleHandle` recorriendo la lista `InMemoryOrderModuleList`. Literal (extracto) de `crates/utils/src/lib.rs`:

```
#[allow(non_snake_case)]

pub fn GetModuleHandle(ModuleName: Option<&str>)-> Option<Arc<Mutex<HMODULE>>>
{
```

```
// Esto es un vector de modulos ya encontrados.
let mut modules_array  = match MODULES.lock(){
    Ok(p) => { p},
    Err(err) => {
        print!("Error With Locking module array: {}",err);
        return None
    }

};


// Iteramos el vector y obtenemos el lock del mutex
for module_mutex in modules_array.iter(){
    let module = match module_mutex.lock(){
        Ok(p) => { p},
        Err(err) => {
            print!("Error with locking hmodule {}",err);
            return None
        }

    };


// Buscamos si ya tenemos el modulo guardado
        match ModuleName {
            Some(name) => if module.Name.eq_ignore_ascii_case(name){ return
Some(Arc::clone(&module_mutex))}
            None => if module.Name.contains("BaseImage") {return
Some(Arc::clone(&module_mutex))}
        }

    }
// A partir de aqui inicia realmente el PEB Walking


// Accedemos el PEB Walking con la funcion readgsqword(0x60) y la convertimos
a un puntero hacia el PEB
let MyPeb: *const PEB = __readgsqword(0x60) as *const PEB;
//Si no se pasa ningun nombre a esta funcion asumimos que quiere el address
del propio binario.
if ModuleName.is_none(){

    unsafe{
        // Colocamos (*MyPeb).ImageBaseAddress en el Addr del BaseImage
        let base_module: HMODULE = HMODULE { Name: "BaseImage".to_string(),
Addr: (*MyPeb).ImageBaseAddress as usize, pe: None };
```

```
        let arc_module= Arc::new(Mutex::new(base_module));

        modules_array.push(arc_module.clone());

        return Some(arc_module);

    }

}

//Aqui obtenemos una referencia de Ldr del PEB
let pLDR: &PEB_LDR_DATA = unsafe{ &*(*MyPeb).Ldr};
//Accedemos el InMemoryOrderModuleList
let List: &LIST_ENTRY = &(*pLDR).InMemoryOrderModuleList;
// Accedemos al Flink
let pFirst: &LIST_ENTRY = unsafe {&*(*List).Flink};
let mut iterator: &LIST_ENTRY = pFirst;

//Colocamos 16 como lsize este size se utilizara para restarselo al iterador y
acceder al LDR_DATA_TABLE_ENTRY
let lsize: usize = 16;
while iterator != List {
// Aqui al iterador le quitamos el lsize y estamos en el LDR_DATA_TABLE_ENTRY
let pEntry: *const LDR_DATA_TABLE_ENTRY = ((iterator as *const LIST_ENTRY) as
usize - lsize) as *const LDR_DATA_TABLE_ENTRY;

unsafe {
//Utilizando la implementacion que creamos de UNICODE_STRING para convertilo a
un string utf8 convertimos el BaseDLLName
let toutf8: String = match (*pEntry).BaseDllName.to_string(){
        Some(utf8) => utf8,
        None =>{ println!("Something happened here");
    return None}

};

// Comparamos que el nombre de Modulo que buscamos coincida y lo agregamos a
los modulos
if toutf8.eq_ignore_ascii_case(ModuleName.unwrap_or_default()) {
    let current_module: HMODULE = HMODULE{
    Name:  toutf8,
    Addr: (*pEntry).DllBase as usize,
    pe: None
    };
```

```
        let arc_module= Arc::new(Mutex::new(current_module));
        modules_array.push(arc_module.clone());
        return Some(arc_module);
    }

    }
    // Si no pasamos al otro modulo.
    iterator = unsafe{&*(*iterator).Flink};

    }

    return None;

    }
```

Puntos a enfatizar:

- Hay un offset hardcodeado `lsize: usize = 16;` para llegar desde `LIST_ENTRY` a `LDR_DATA_TABLE_ENTRY`.
- En C usarías `CONTAINING_RECORD`, aquí está “manual”.

---

5) Resolver dinámico: `GetPESections` y `GetProcAddress`

5.1 `GetPESections`

`crates/utils/src/lib.rs`:
Esta función se encarga de parsear las cabeceras PE del módulo en memoria y guardar los punteros a sus estructuras clave.

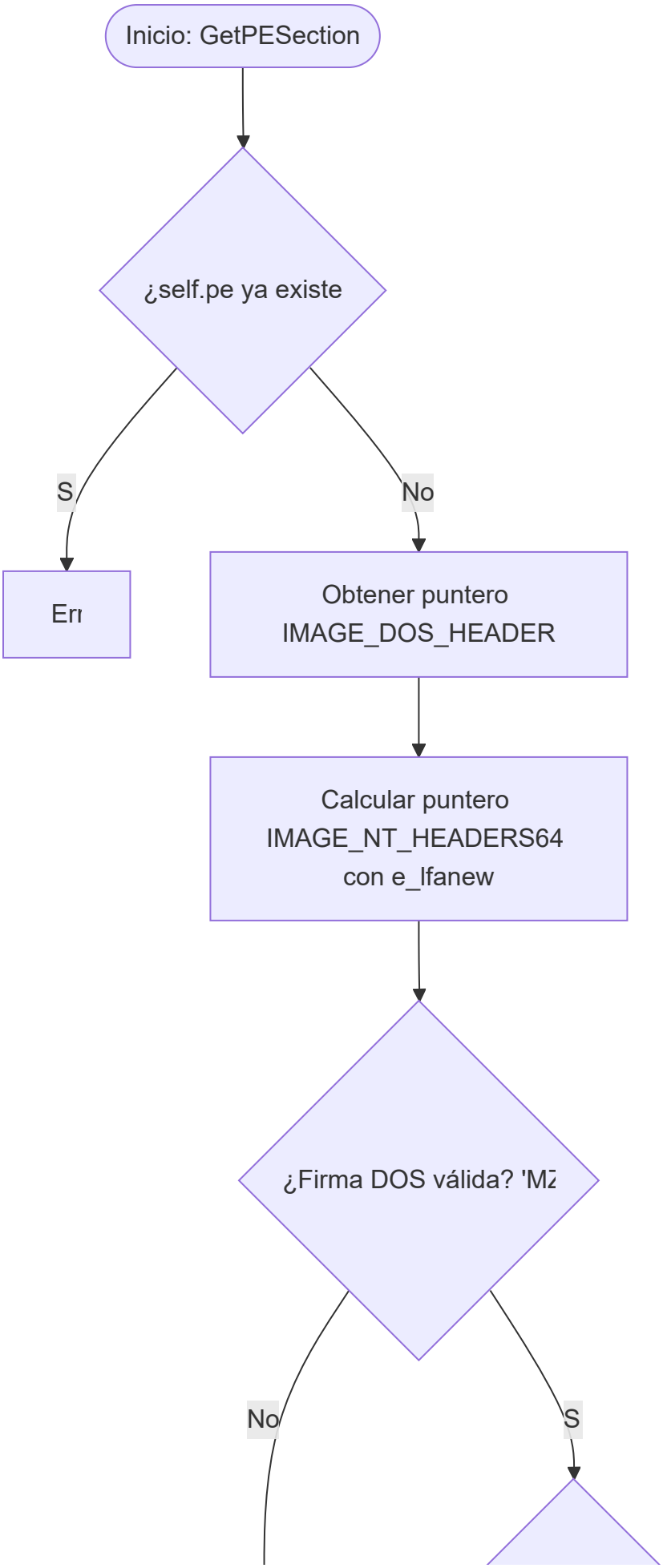
Utiliza la

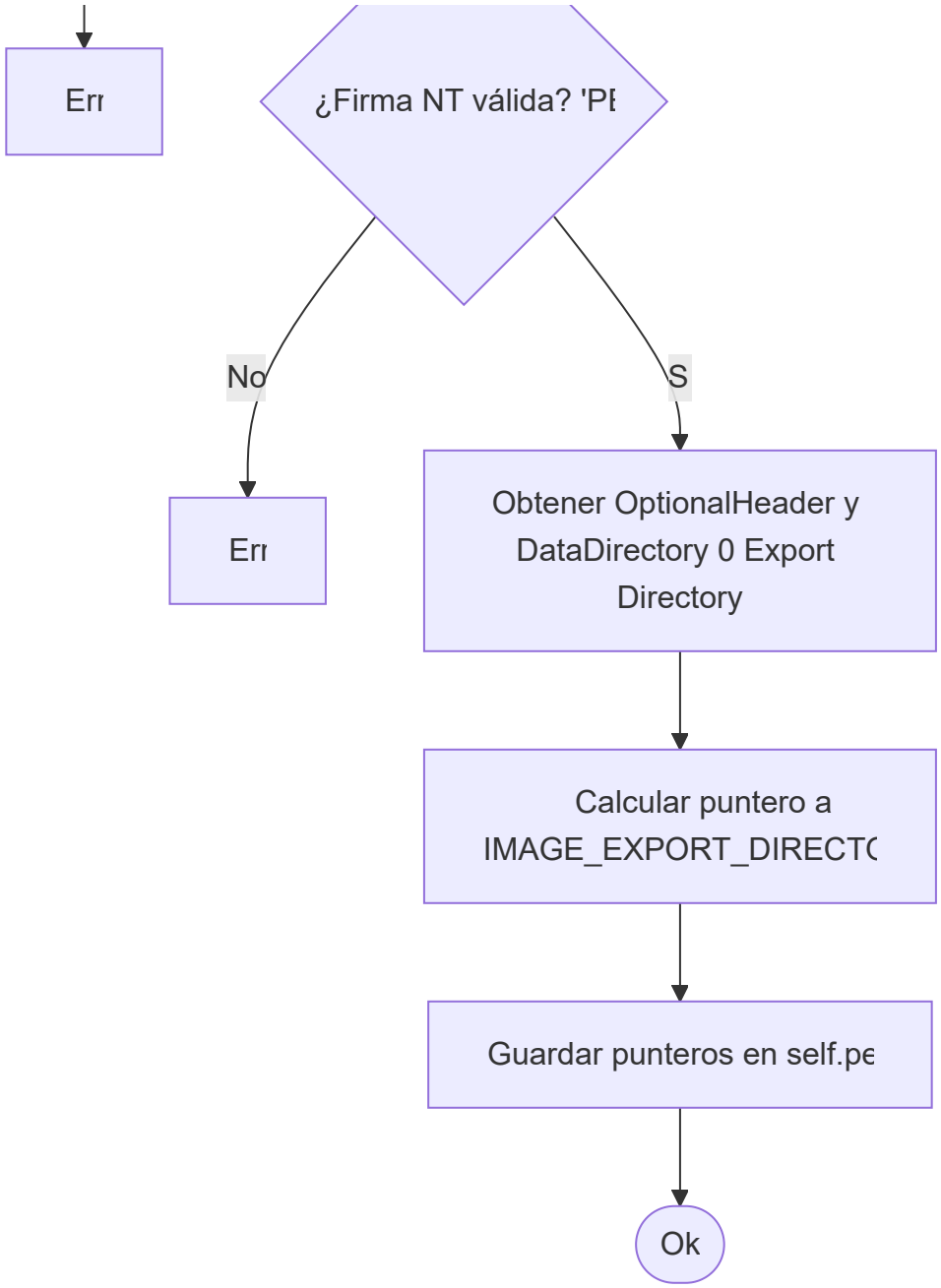
Desglose paso a paso:

1. **Comprobación Inicial**: Verifica si la estructura `pe` ya ha sido parseada (`self.pe.is_some()`). Si es así, retorna `Err(false)` para evitar trabajo redundante.
2. **Cabecera DOS**: Calcula el puntero a la estructura `IMAGE_DOS_HEADER` a partir de la dirección base del módulo (`self.Addr`).
3. **Cabecera NT**: Utiliza el campo `e_lfanew` de la cabecera DOS para encontrar y calcular el puntero a la cabecera `IMAGE_NT_HEADERS64`.
4. **Validación de Firmas**:

- Comprueba el `e_magic` de la cabecera DOS (debe ser `IMAGE_DOS_SIGNATURE` o "MZ"). Falla y retorna `Err(false)` si es inválido.
- Comprueba la `Signature` de la cabecera NT (debe ser `IMAGE_NT_SIGNATURE` o "PE\0\0"). Falla y retorna `Err(false)` si es inválido.

5. **Directorios de Datos**: Localiza el `OptionalHeader`, extrae el primer `DataDirectory` (que corresponde al Directorio de Exportaciones) y calcula su dirección virtual (VA).
6. **Guardado de Estado**: Encapsula todos estos punteros y los guarda en la variable `self.pe`.
7. **Éxito**: Retorna `Ok(true)`.


Inicio: GetPESection
¿self.pe ya existe
S
No
Err
Obtener puntero
IMAGE_DOS_HEADER
Calcular puntero
IMAGE_NT_HEADERS64
con e_lfanew
¿Firma DOS válida? 'MZ
No
S




```
impl HMODULE {
#[allow(non_snake_case)]

pub fn GetPESections(&mut self)->Result<bool,bool>{
    if self.pe.is_some(){
        return Err(false);
    }
//Utilizamos el base address y lo casteamos a IMAGE_DOS_HEADER
    let dos_pointer: *const IMAGE_DOS_HEADER =  self.Addr as *const
IMAGE_DOS_HEADER;
    //Esto es opcional simplemente de un puntero raw lo estoy volviendo una
referencia en rust.
    let pIMAGE_DOS_HEADER: &IMAGE_DOS_HEADER = unsafe{ &*dos_pointer};
```

```
    // Aqui queremos acceder al NT_HEADER por lo que sumando el base address +
e_lfanew llegamos al NT_HEADER
    let nt_pointer: *const IMAGE_NT_HEADERS64 = RVA(self.Addr ,
(*pIMAGE_DOS_HEADER).e_lfanew as usize) as *const IMAGE_NT_HEADERS64;
    //Nuevamente es opcional pero aqui llegamos lo volvemos una referencia.
    let pIMAGE_NT_HEADERS: &IMAGE_NT_HEADERS64  = unsafe {&*nt_pointer};

//Comprobamos que tanto el e_magic como el Signature del NT_HEADER sean los
correctos para confirmar que estamos haciendo correctamente el parsing.
    if (*pIMAGE_DOS_HEADER).e_magic != IMAGE_DOS_SIGNATURE{
       return Err(false)
    }

    if (*pIMAGE_NT_HEADERS).Signature != IMAGE_NT_SIGNATURE{
      return Err(false)
    }


//Obtenemos la ubicacion del OPTIONAL_HEADER
    let pOptionalHeader: &IMAGE_OPTIONAL_HEADER64 = &
(*pIMAGE_NT_HEADERS).OptionalHeader;
    let optional_pointer: *const IMAGE_OPTIONAL_HEADER64 = &
(*pIMAGE_NT_HEADERS).OptionalHeader as *const IMAGE_OPTIONAL_HEADER64;
// Obtenemos la direccion del Data Directory en indice 0 que es el
EXPORT_DIRECTORY
    let data_pointer: *const IMAGE_DATA_DIRECTORY = &
(*pOptionalHeader).DataDirectory[0] as *const IMAGE_DATA_DIRECTORY;
    let pDataDirectory: &IMAGE_DATA_DIRECTORY = &
(*pOptionalHeader).DataDirectory[0];


//Aqui utilizando el VA del DataDirectory + base address obtenemos la
direccion absoluta del Export Directory
    let Export_pointer: *const IMAGE_EXPORT_DIRECTORY = RVA(self.Addr ,
pDataDirectory.VirtualAddress as usize) as *const IMAGE_EXPORT_DIRECTORY;



//Almacenamos toda la informacion que hemos conseguido.
    self.pe = Some(PE {
    dos_header: NonNull::new(dos_pointer as *mut IMAGE_DOS_HEADER).unwrap(),

    nt_header: NonNull::new(nt_pointer as *mut IMAGE_NT_HEADERS64).unwrap(),

    optional_header: NonNull::new(optional_pointer as *mut
IMAGE_OPTIONAL_HEADER64).unwrap(),
```

```
    data_directory: NonNull::new(data_pointer as *mut
IMAGE_DATA_DIRECTORY).unwrap(),

    export_directory: NonNull::new(Export_pointer as *mut
IMAGE_EXPORT_DIRECTORY).unwrap() });

   return Ok(true);

}



}
```

5.2 `GetProcAddress`

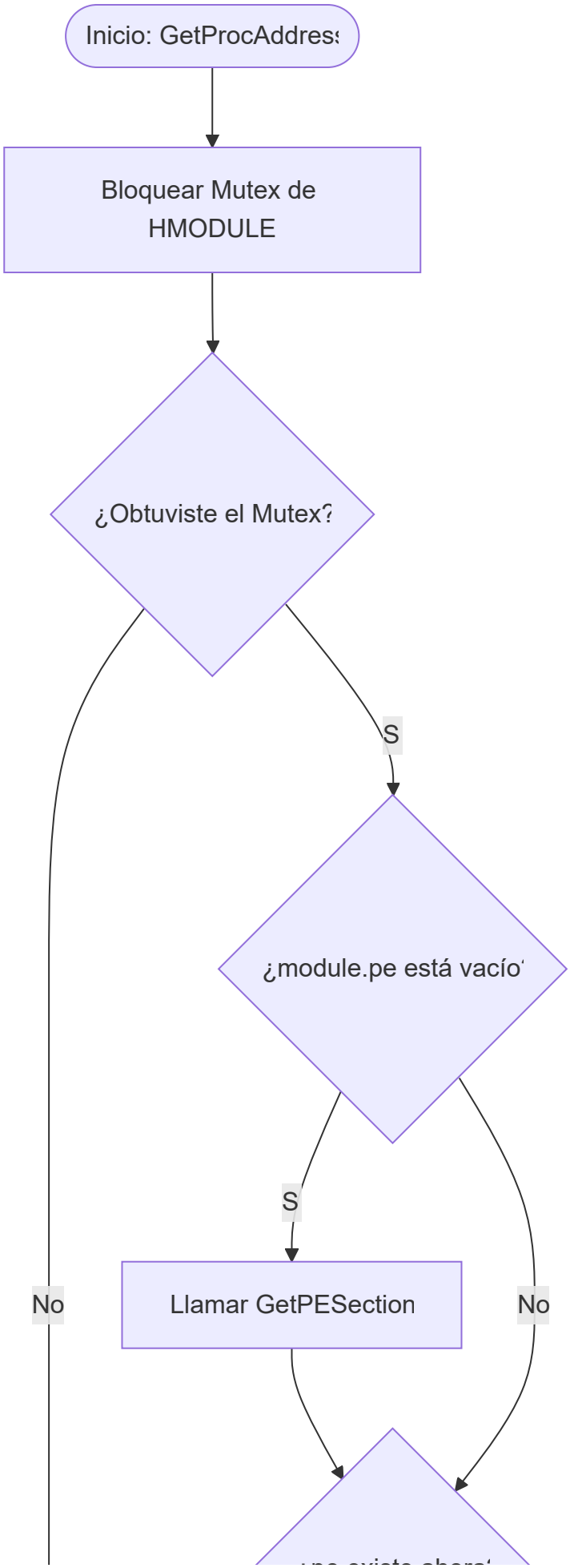
`crates/utils/src/lib.rs`:

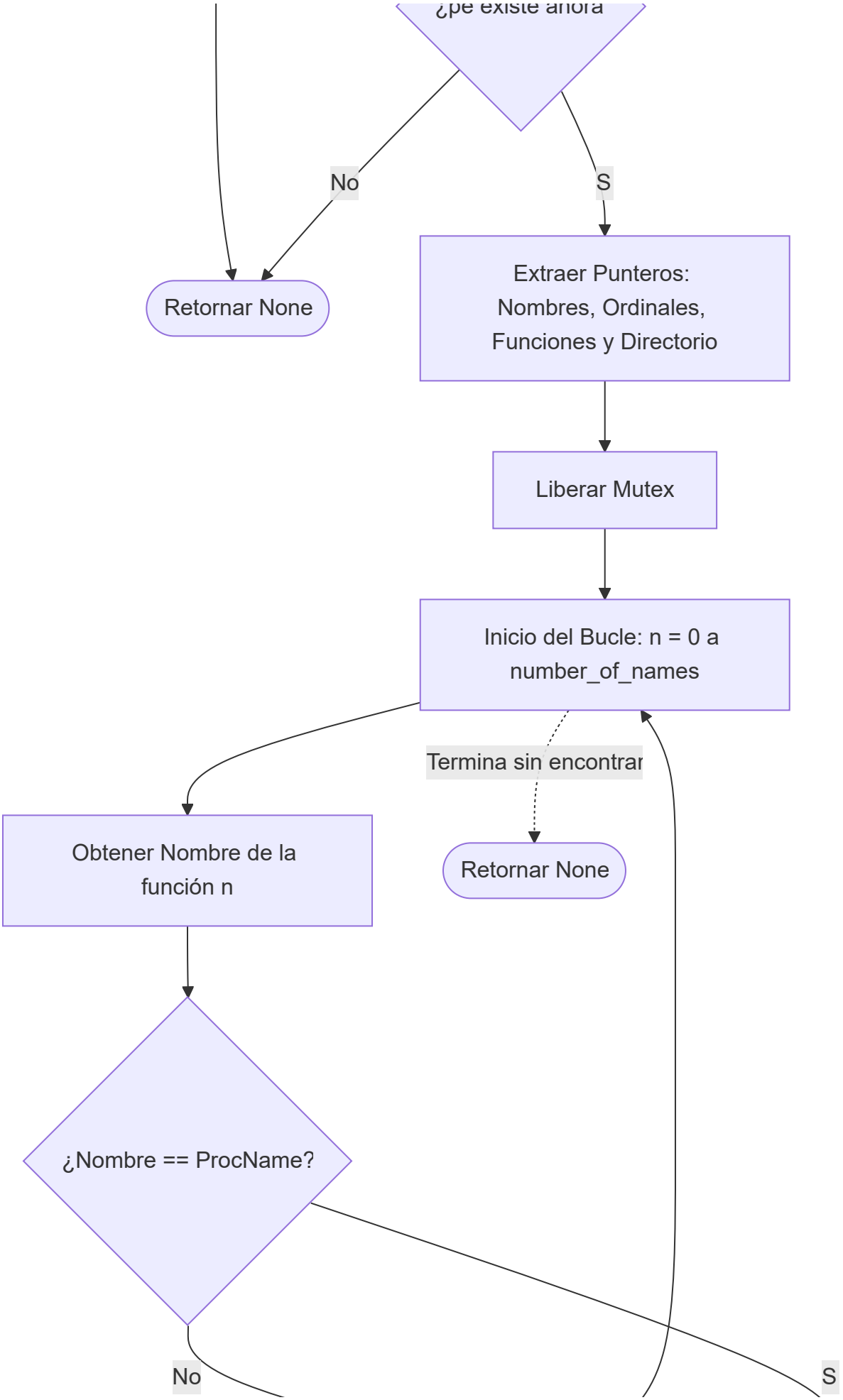
Desglose paso a paso:

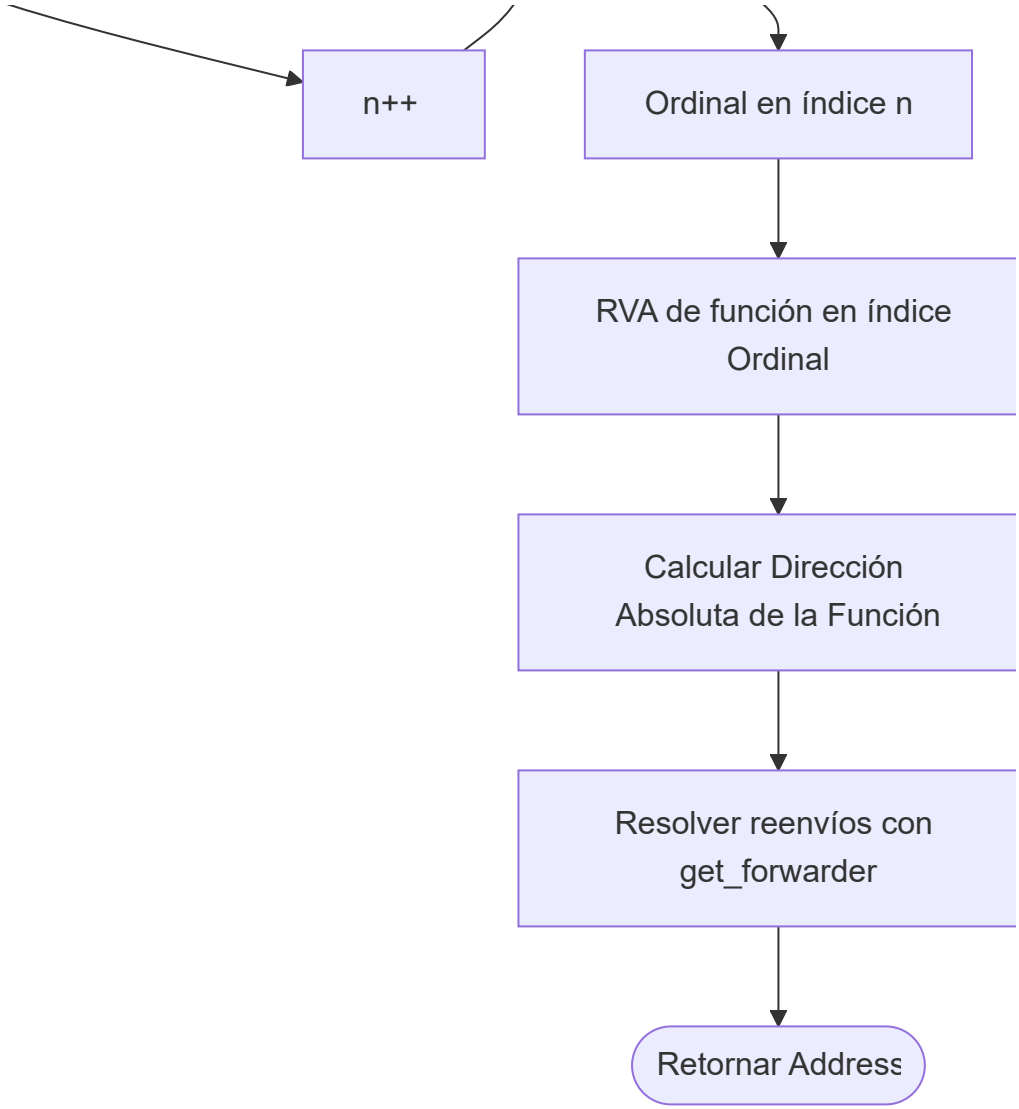
1. **Sincronización de Hilos**: Bloquea el `Mutex` que protege al `HMODULE` para asegurar el acceso seguro al estado interno. Si el bloqueo falla, retorna `None`.
2. **Inicialización Perezosa (Lazy Init)**: Verifica si el módulo ya parseó sus cabeceras PE (`module.pe.is_none()`). Si no lo ha hecho, llama a `GetPESections()`.
3. **Extracción de Tablas de Exportación**:
   - Resuelve los punteros principales del directorio de exportaciones: Tabla de nombres (`AddressOfNames`), Tabla de ordinales (`AddressOfNameOrdinals`) y Tabla de funciones (`AddressOfFunctions`).
   - Obtiene el número total de nombres exportados y los detalles del directorio de datos.
   - Libera el bloqueo del mutex al salir del bloque de variables (implícito).
4. **Bucle de Búsqueda de Nombres**: Itera a través de las funciones exportadas (desde `0` hasta `number_of_names`):
   - Extrae el nombre de la función calculando su RVA (Relative Virtual Address) y convirtiéndolo a un string en Rust.
   - Compara el nombre extraído con `ProcName`.
5. **Resolución de Dirección (Si hay coincidencia)**:
   - Extrae el número **Ordinal** asociado al nombre coincidente.
   - Usa ese Ordinal como índice en la Tabla de Funciones (`AddressOfFunctions`) para obtener el RVA de la función.
   - Calcula la **dirección absoluta** sumando la base del módulo al RVA.

6. **Manejo de Forwarders**: Llama a `get_forwarder()` para resolver funciones reenviadas (funciones que apuntan a otras DLLs). Si la función no está reenviada, usa la dirección calculada previamente.
7. **Retorno**: Si la función coincide, imprime la ruta en modo *Debug* y retorna `Some(funcaddr)`. Si finaliza el bucle y no se encontró la función, retorna `None`.

Diagrama de Flujo Visual


Inicio: GetProcAddress
Bloquear Mutex de HMODULE
¿Obtuviste el Mutex?
S
¿module.pe está vacío
S
No
Llamar GetPESection
No


¿pe existe ahora
No
S
Retornar None
Extraer Punteros:
Nombres, Ordinales,
Funciones y Directorio
Liberar Mutex
Inicio del Bucle: n = 0 a
number_of_names
Termina sin encontrar
Retornar None
Obtener Nombre de la
función n
¿Nombre == ProcName?
No
S




```
pub fn GetProcAddress(hmodule: Arc<Mutex<HMODULE>> ,ProcName: &str) ->
Option<*const c_void>{
    //Inicializar variables
    let mut address = 0;
    let pName: *const u32;
    let pOrdinals: *const u16;
    let pFuncs: *const u32;
    let number_of_names: u32;
    let mut ddirectory_va = 0;
    let mut ddirectory_size = 0;

    {
    //Obtener el mutex al HMODULE
    let p = hmodule.lock();

    if p.is_err(){

        return None;
```

```
    }
    // Si el HMODULE aun no se ha parseado las secciones llamamos
GetPESections
    let mut module = p.unwrap();
    if module.pe.is_none(){
            module.GetPESections();
    }

    let ppe = match &module.pe {
        Some(pe) => pe,
        None => return None
    };
    //Obtenemos el base address del modulo
    address = module.Addr;
    // Obtenemos la direccion absoluta de AddressOfNames
    pName = unsafe {RVA(address,ppe.export_directory.as_ref().AddressOfNames
as usize) as *const u32};
    // Obtenemos la direccion absoluta de AddressOfOrdinals
    pOrdinals = unsafe {RVA(address ,
ppe.export_directory.as_ref().AddressOfNameOrdinals as usize) as *const u16};
    //Obtenemos la direccion absoluta de AddressOfFuctions
    pFuncs= unsafe {RVA(address ,
ppe.export_directory.as_ref().AddressOfFunctions as usize) as *const u32};
    //Obtenemos la cantidad de nombres almacenados
    number_of_names = unsafe {ppe.export_directory.as_ref().NumberOfNames};
    //Obtenemos el VA del DataDirectory numero 0
    ddirectory_va = unsafe {ppe.data_directory.as_ref().VirtualAddress as
usize};
    // Obtenemos el size de este data directory
    ddirectory_size =  unsafe {ppe.data_directory.as_ref().Size as usize};




}




    unsafe{
    // Bucle desde 0 hasta el numero de nombres almacenados
    for n in 0..number_of_names{
   // Dentro del AddressOfNames se almacenan los VA's hacia el nombre asi que
luego de obtener el VA hay que obtener la direccion absoluta del nombre.
            let funcname = core::ffi::CStr::from_ptr((address + *pName.add(n
as usize) as usize) as *const i8);
//Una vez tenemos el location del nombre lo convertimos a un str
```

```
            let str = funcname.to_str().unwrap_or_default();
//Comparamos si es la funcion que buscamos
            if str == ProcName {
//Si lo es entonces utilizando el mismo indice utilizado en el AddressOfNames
accedemos el AddressOfOrdinals en ese indice y obtendremos el ordinal
correcto, este ordinal es el indice a utilizar dentro del AddressOfFunctions
para obtener el VA hacia la funcion que se busca.
            let funcaddr : *const c_void = (address +
*pFuncs.add(*pOrdinals.add(n as usize) as usize) as usize) as *const c_void;
//Aqui comprobamos si el address que obtuvimos en realmente una funcion o un
forwarder.
                let funcaddr = match get_forwarder(address
,ddirectory_va,ddirectory_size,funcaddr){
                    Ok(p)=>p,
                    Err(_)=>funcaddr
                };

                #[cfg(debug_assertions)]{
                print!("RVA to Function: {:x}\n",*pFuncs.add(*pOrdinals.add(n
as usize) as usize) as u32);
                print!("Function {:p}\n",funcaddr);

            }
//Retonarmos el address de la funcion
            return Some(funcaddr)

            }

            }

        }

        return None;

    }

}
```

---

6) Validación rápida

Desde la raíz:

```
cargo test -p get-proc
cargo run -p get-proc --bin whoami
```

---

7) Troubleshooting

- **Offsets hardcodeados** (`lsize = 16`) ⇒ fragilidad.
- **UTF-16**: `Length` en bytes.
- **RVA vs VA**: siempre sumar el base.
- **x64 vs x86**: cambia GS/FS y cambian layouts.
- mutables y no mutables si no asignamos una variable como mut no podemos modificarla

```
let mut variable;
let variable;
```